



COMP3055

Machine Learning

Topic 11 – Perceptron, ADLINE, and Delta Rule

Zheng Lu
2025 Autumn

Recall: Machine Learning Concept

- Artificial Intelligence: AI is the science of making machines do things that require intelligence if done by men (Minsky 1986). 完成人咧完成需要智能的任务
- Machine Learning is an area of AI concerned with development of techniques which allow machines to learn.
- Why Machine Learning?
 - To build machines exhibiting intelligent behaviour (i.e., ^{能够推理, 预测和适应} able to reason, predict, and adapt) while helping humans work, study, and entertain themselves

人工智能是“让机器具备智能”，而机器学习是“让机器自己学会智能”的核心手段。

Machine Learning Algorithms

- The success of machine learning system depends on the algorithms. 机器学习的成败取决于算法
- The algorithms control the search to find and build the knowledge structures. 算法控制模型搜索，以找到和构建知识结构
- The learning algorithms should extract useful information from training examples. 学习算法应该从训练样本数据中提取有用信息

Machine Learning Algorithms

- 有标签 • **Supervised learning** (generates a function that maps inputs to desired outputs) 监督学习：生成一个函数可以把输入映射到期望的输出
 - **Prediction** 预测：根据已有数据预测未来
 - **Classification (discrete labels), Regression (real values)**
分类（离散类别） 回归（输出数值）
- 无标签 • **Unsupervised learning** (models a set of inputs, labelled examples are not available) 无监督学习：接受一组数据，未标签数据
 - **Probability distribution estimation** 概率分布估计
 - **Finding association (in features)** 特征关联：找出特征间相关性
 - **Dimension reduction** 降维：压缩高维数据，保留主要信息（PCA）
 - **Clustering** 聚类：相似样本分成组
- 通过反馈 • **Reinforcement learning** (learning by education / experience) 强化学习：基于教育/经验的学习方式
 - **Decision making (robot, chess machine)** 决策制定（机器人/下棋程序）

Artificial General Intelligence

通用人工智能

假想的机器智能

- **Artificial General Intelligence (AGI)** is the hypothetical intelligence of a machine that has the capacity to understand or learn any intellectual task that a human being can. AGI can also be referred to as **strong AI**

有与人类相同的理解和学习能力

- Example: **Turing Test** 图灵测试

图灵测试 是检验机器是否具有人类智能的经典方法。

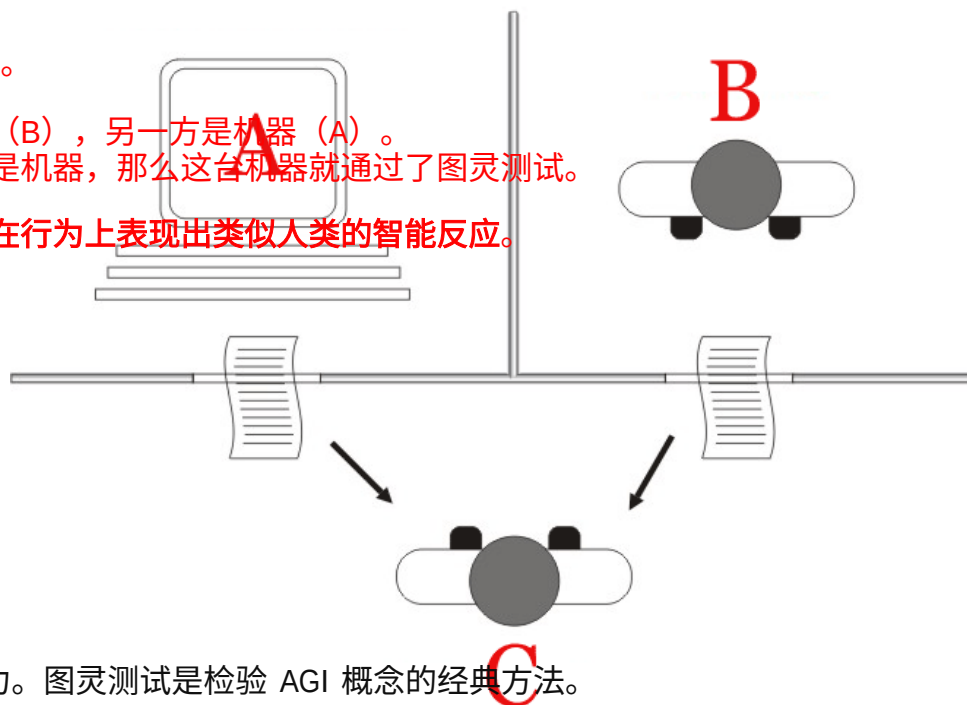
测试方式：

一位人类评审员通过文字与两方交谈：一方是人类（B），另一方是机器（A）。

如果评审员无法可靠地区分哪一方是人类、哪一方是机器，那么这台机器就通过了图灵测试。

意义：

图灵测试不要求机器“真正理解”世界，只要求能在行为上表现出类似人类的智能反应。



AGI = 强人工智能 (Strong AI)

目标是让机器具备与人类相当的认知能力与学习能力。图灵测试是检验 AGI 概念的经典方法。

Purposeful Problem Solver

- In contrast to strong AI, **weak AI** is not intended to perform human cognitive abilities, rather, **weak AI is limited to the use of software to study or accomplish specific problem solving or reasoning tasks.**

与强人工智能不同，弱人工智能并非旨在执行人类的认知能力，而是仅限于利用软件来研究或完成特定的问题解决或推理任务。

- Examples: **face recognition, speech recognition, games, etc.**

人脸识别，语音识别，游戏

- Application domains: **security, medicine, education, finances, genetics, etc.**

安全，医疗，级哦啊鱼，金融，遗传学

Weak AI = Task-specific AI (任务型人工智能)

它专注于在特定领域高效完成任务，但不具备像人一样的通用智能或意识。

Feature Engineering 特征工程

- Feature engineering is the process of using domain knowledge to extract features from raw data via data mining techniques. These features can be used to improve the performance of machine learning algorithms.

特征工程是利用领域知识，通过数据挖掘技术从原始数据中提取特征的过程。这些特征可用于提升机器学习算法的性能。

二、为什么重要 (Why it matters)

- 原始数据往往杂乱无章，算法难以直接学习。

- Process:
 - Brainstorming or testing features; 开展头脑风暴或测试特征
 - Deciding what features to create; 决定创建哪些特征
 - Creating features; 创建特征
 - Checking how the features work with your model; 检查这些特征与模型的配合效果
 - Improving your features if needed; 如有必要则改进特征
 - Go back to brainstorming/creating more features until the work is done. 重复进行头脑风暴/创建更多特征，直至工作完成

- 高质量的特征能：
 - 提高模型准确率；
 - 减少训练时间；
 - 改善泛化性能；
 - 让简单算法也能获得好结果。
- 一句话总结：好特征 > 好模型。

特征工程是一个循环优化的过程，而不是一次性任务。

Feature Engineering = 用领域知识把数据“变聪明”。
它是机器学习中连接“原始数据”和“算法性能”的关键桥梁。

Perceptron - Basic

Perceptron = 最简单的人工神经元模型它通过计算加权输入的总和并应用阈值函数，实现二分类决策（+1 / -1）。

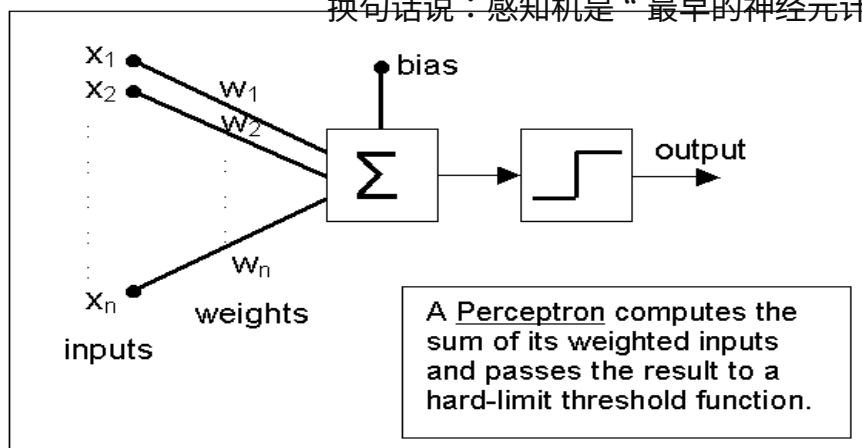
Perceptron is a type of Artificial Neural Network (ANN)

Perceptron（感知机）是一种最基本的人工神经网络模型（ANN）

它模拟了**生物神经元（Neuron）**的行为：

接收输入-->计算加权和-->应用激活函数-->输出结果。

换句话说：感知机是“最早的神元计算模型”，也是神经网络的原型。



元件	含义	作用
Inputs (输入) $x_1, x_2, \dots, x_n$	输入特征值	模型接收的原始数据
Weights (权重) $w_1, w_2, \dots, w_n$	每个输入的重要性	决定输入对输出的影响程度
Bias (偏置) b	常数项	用于平移决策边界 (相当于截距)
Σ (求和器)	加权求和器	计算 $R = \sum w_i x_i + b$
Activation / Threshold Function (激活函数/阈值函数)	决定输出	将线性结果转化为离散输出 (如 +1 或 -1)
Output (输出)	最终决策	模型预测的结果

1. 接收输入：每个输入 x_i 乘以相应权重 w_i 。

2. 线性组合：求加权和 + 偏置：

$$R = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$$

3. 激活函数处理：

• 若 $R > 0$ ，输出 1；

• 若 $R \leq 0$ ，输出 -1。

（即硬限制函数 Hard-limit / Step Function）

4. 输出结果：产生最终分类结果。

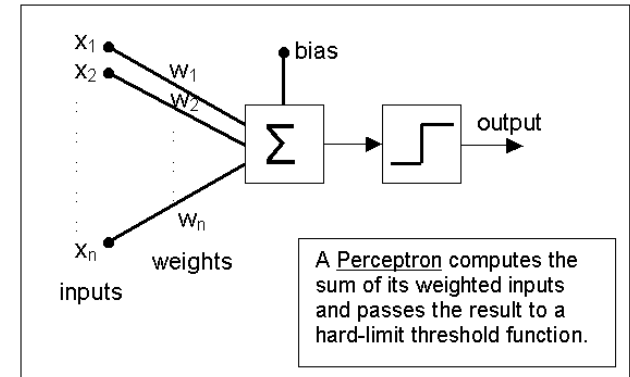
• 感知机可看作一个线性分类器 (linear classifier)，
它试图找到一条“决策边界 (decision boundary)”，将数据分为两类。

• 决策边界的方程为：

$$w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b = 0$$

Perceptron - Operation

Perceptron takes a vector of real-valued inputs, calculates a **linear combination** of these inputs. Then it outputs **1** if the result is greater than some **threshold** and **-1** otherwise.



- x_i : 输入特征;
- w_i : 对应的权重;
- w_0 : 偏置 (bias) 或常数项;
- R : 线性求和值 (即神经元的输入信号强度)。

$$R = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n = w_0 + \sum_{i=1}^n w_ix_i$$

$$output = o = sign(R) = \begin{cases} +1, & \text{if } R > 0 \\ -1, & \text{otherwise} \end{cases}$$

Perceptron – Decision Surface 决策面

- Perceptron can be regarded as representing a hyperplane decision surface in the n -dimensional feature space of instances.

Perceptron可以被看作在 n 维特征空间中形成一个超平面 (hyperplane)。
这个超平面将不同类别的数据点分开，作为模型的决策边界 (decision boundary) 或 决策面 (decision surface)

- The perceptron outputs a 1 for instances lying on one side of the hyperplane and a -1 for instances lying on the other side.

感知机输出 +1：表示样本点位于超平面的一侧。

感知机输出 -1：表示样本点位于另一侧。

因此，感知机的任务就是找到一条（或一个）最佳的“分割线 / 面”，让正类与负类尽可能被分隔开。

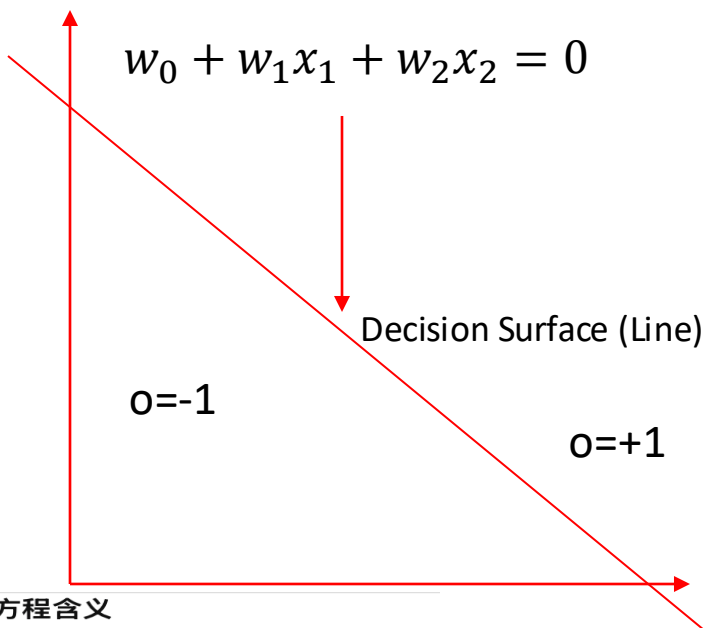
- This hyperplane is called the Decision Surface.

超平面被称为决策面

Perceptron – Decision Surface

在二维空间中 (n=2) 它是一条直线，是二维空间中的决策边界

In 2-dimensional space:

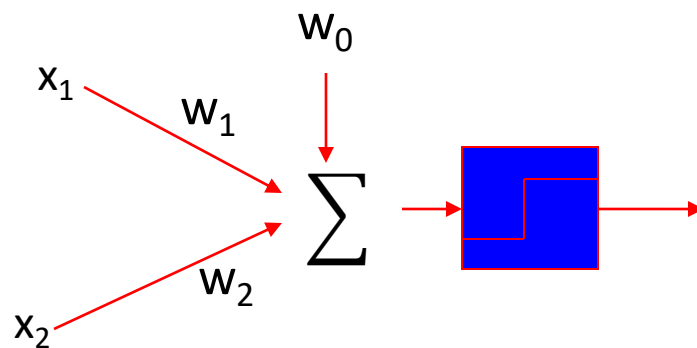


1 直线方程含义

- w_1, w_2 : 控制直线的方向 (斜率);
- w_0 : 控制直线的位置 (截距)。

这条直线把空间划分为两个区域:

$$\begin{cases} R = w_0 + w_1x_1 + w_2x_2 > 0 & \Rightarrow o = +1 \\ R = w_0 + w_1x_1 + w_2x_2 < 0 & \Rightarrow o = -1 \end{cases}$$



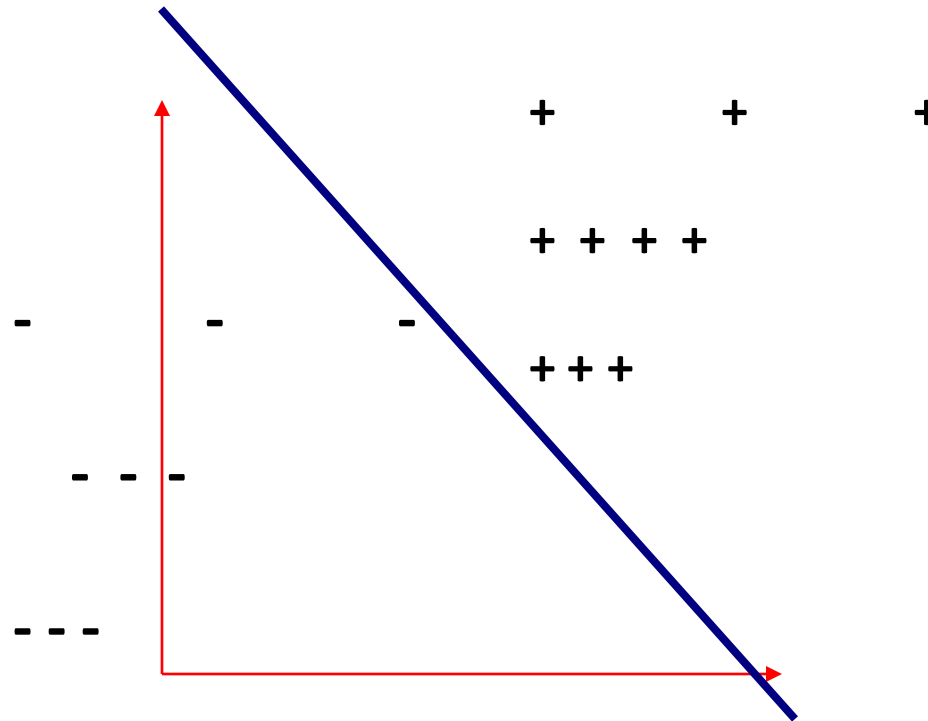
• 右图:

• 展示了感知机的计算流程:

• 输入 $x_1, x_2 \rightarrow$ 加权求和 (Σ) $\rightarrow$ 加上偏置 $w_0 \rightarrow$ 经过激活函数 (阈值函数) $\rightarrow$ 输出 o 。

Perceptron – Representation Power 表示能力

- The Decision Surface is **linear**. 线性
- Perceptron can only solve 线性可分问题 **Linearly Separable Problems**.



Perceptron – Representation Power

布尔函数

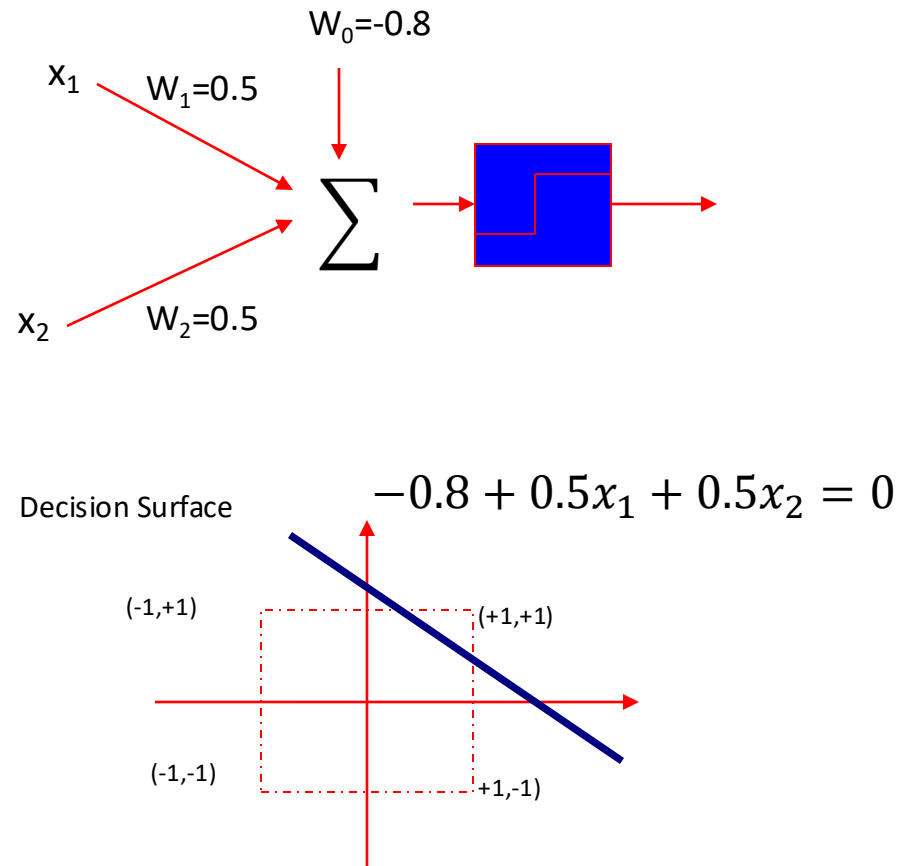
假设布尔值：1=true；-1=false

Can represent many Boolean functions, assuming Boolean values of 1 (true) and -1 (false).

AND

x1	x2	D
-1	-1	-1
-1	+1	-1
+1	-1	-1
+1	+1	+1

只有当两个输入都为真（+1）时，输出才为真（+1）；
其他情况输出都是假（-1）。



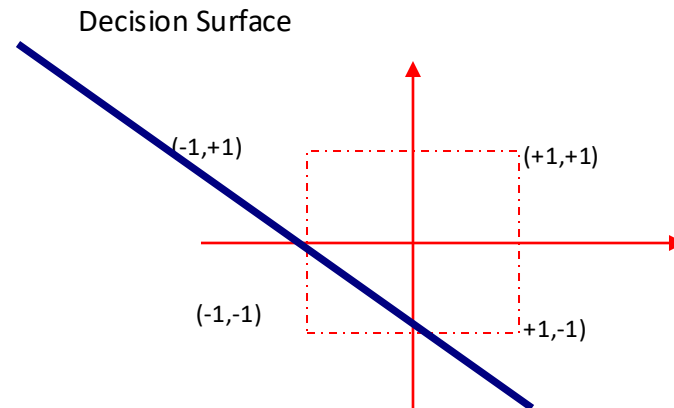
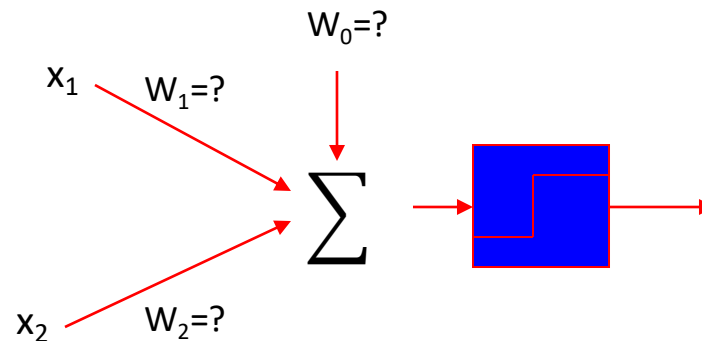
Perceptron – Representation Power

Can represent many Boolean functions, assuming Boolean values of 1 (true) and -1 (false).

OR

x1	x2	D
-1	-1	-1
-1	+1	+1
+1	-1	+1
+1	+1	+1

只要任意一个输入为 True (+1) , 输出就是 True (+1) ;
只有两个输入都为 False (-1) 时, 输出才为 False (-1) 。



Perceptron – Representation Power

Some problems are **linearly non-separable**.

线性不可分

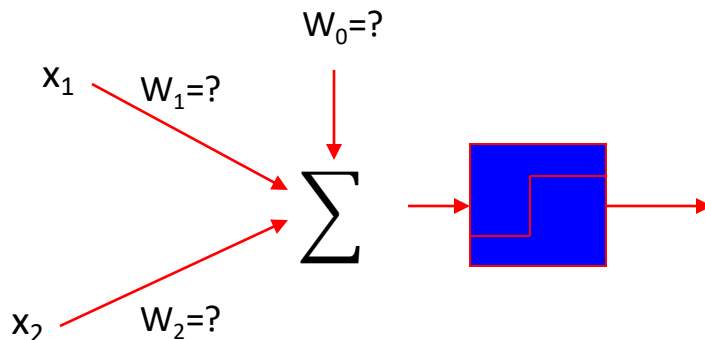
XOR 数据分布在二维平面上呈 **对角线交叉** 的形式：
正样本 (+1)：在两个对角点 (1, +1) 和 (+1, 1)；
负样本 (-1)：在另两个对角点 (1, 1) 和 (+1, +1)。

无论怎样放置一条直线（决策边界）：
都无法把两个正样本与两个负样本完全分开。
换句话说，任何线性超平面都无法把 XOR 数据正确分类

XOR

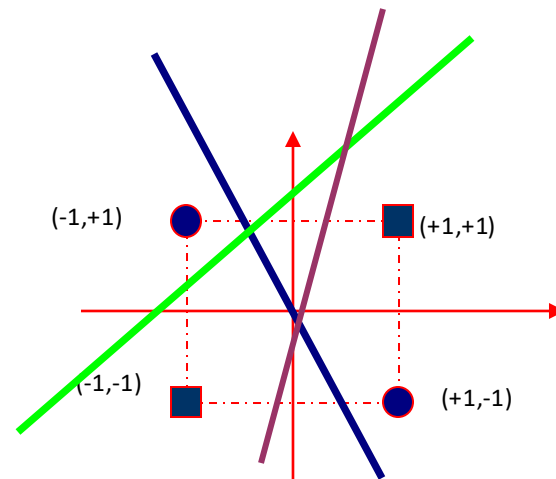
x1	x2	D
-1	-1	-1
-1	+1	+1
+1	-1	+1
+1	+1	-1

当两个输入不同（即“异”）时输出 True (+1)；
当两个输入相同（即“同”）时输出 False (-1)。



Decision Surface:

It doesn't matter where you place the line (decision surface). It is impossible to separate the space such that **on one side we have $o = 1$ and on the other we have $o = -1$.**



Perceptron Cannot Solve such Problem!

Perceptron – Training Algorithm 训练算法

感知机训练是一个迭代过程：

每次输入一个样本、计算输出、若出错则更新权重，直到所有样本被正确分类或算法收敛为止。

Training sample pairs (X, d) , where X is the input vector, d is the input vector's classification (+1 or -1) is iteratively presented to the network for training, one at a time, until the process converges.

迭代输入
逐个样本
收敛

二、训练数据的组成 (Training Sample Pairs)

每个训练样本由一对数据组成：

(X, d)

其中：

- $X = [x_1, x_2, \dots, x_n]$: 输入向量 (features)
- d : 输入的目标标签 (desired output), 通常取 +1 (正类) 或 -1 (负类)

三、训练过程 (Training Procedure)

1. Iteratively presented (迭代输入)

- 每次从训练集中取出一个样本 (X, d) 。
- 将其输入感知机模型进行计算输出。

2. One at a time (逐个样本)

- 感知机的更新是在线 (online) 学习方式，即每次处理一个样本后立刻更新权重。

3. Until convergence (直到收敛)

- 重复以上步骤，直到所有样本都被正确分类 (或误差不再变化)。
- “收敛 (converges)” 意味着模型权重已稳定，不再发生大的调整。

Perceptron – Training Algorithm

The Procedure is as follows:

初始化权重：将权重设置为 $(-1, 1)$ 之间较小的随机数值；随机可以避免陷入对称状态

1. Set the weights to small random values, e.g., in the range $(-1, 1)$
2. Present X , and calculate

$$R = \sum_{i=0}^n w_i x_i, \quad o = \text{sign}(R) = \begin{cases} +1, & \text{if } R > 0 \\ -1, & \text{otherwise} \end{cases}$$

let $x_0=1$

当输出错误时 (即 $o \neq d$):

$$w_i \leftarrow w_i + \eta(d - o)x_i, \quad i = 0, 1, \dots, n$$

1. Update the weights

$$w_i \leftarrow w_i + \eta(d - o)x_i, \quad i = 0, 1, \dots, n$$

$0 < \eta < 1$, is the training rate

循环迭代，直到收敛

2. Repeat by going to step 2

其中:

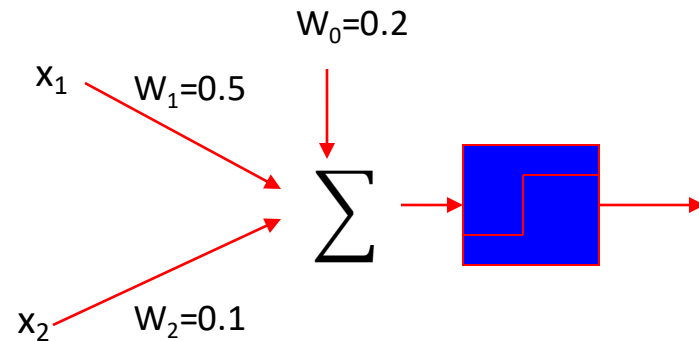
- d : 样本的目标输出;
- o : 模型的预测输出;
- η : 学习率 (training rate), 满足 $0 < \eta < 1$ 。

* 如果分类正确 ($d = o$), 则 $d - o = 0$, 权重不会变化

Perceptron – Training Algorithm

Example

x1	x2	D
-1	-1	-1
-1	+1	+1
+1	-1	+1
+1	+1	+1



$$w_i \leftarrow w_i + \eta(d - o)x_i, i = 1, 2, \dots, n$$

对每个样本依次执行以下步骤：

1 计算加权和：

$$R = w_0 + w_1x_1 + w_2x_2$$

2 根据符号函数输出：

$$o = \text{sign}(R) = \begin{cases} +1, & R > 0 \\ -1, & R \leq 0 \end{cases}$$

3 更新权重（若分类错误）：

$$w_i \leftarrow w_i + \eta(d - o)x_i, \quad i = 0, 1, 2$$

第 1 轮 (Epoch 1)

样本 1: $(x_1, x_2, d) = (-1, -1, -1)$

- $R = 0.2 + 0.5(-1) + 0.1(-1) = -0.4 \Rightarrow o = -1$ (正确)
- 不更新；权重仍： $(w_0, w_1, w_2) = (0.2, 0.5, 0.1)$

样本 2: $(-1, +1, +1)$

- $R = 0.2 + 0.5(-1) + 0.1(+1) = -0.2 \Rightarrow o = -1$ (错误)
- $(d - o) = 1 - (-1) = 2$
- 更新：
 $w_0 \leftarrow 0.2 + 0.1 \cdot 2 \cdot 1 = 0.4$
 $w_1 \leftarrow 0.5 + 0.1 \cdot 2 \cdot (-1) = 0.3$
 $w_2 \leftarrow 0.1 + 0.1 \cdot 2 \cdot (+1) = 0.3$

- 新权重： $(0.4, 0.3, 0.3)$

样本 3: $(+1, -1, +1)$

- $R = 0.4 + 0.3(+1) + 0.3(-1) = 0.4 \Rightarrow o = +1$ (正确)
- 不更新；权重保持： $(0.4, 0.3, 0.3)$

样本 4: $(+1, +1, +1)$

- $R = 0.4 + 0.3(+1) + 0.3(+1) = 1.0 \Rightarrow o = +1$ (正确)
- 不更新；权重保持： $(0.4, 0.3, 0.3)$

第 1 轮结束仍有错误（样本 2），继续下一轮。

第 2 轮 (Epoch 2)

用权重 $(0.4, 0.3, 0.3)$ 重新遍历四个样本：

1. $(-1, -1, -1)$:
 $R = 0.4 + 0.3(-1) + 0.3(-1) = -0.2 \Rightarrow o = -1$ ✓ (不更)
2. $(-1, +1, +1)$:
 $R = 0.4 + 0.3(-1) + 0.3(+1) = 0.4 \Rightarrow o = +1$ ✓ (不更)
3. $(+1, -1, +1)$:
 $R = 0.4 + 0.3(+1) + 0.3(-1) = 0.4 \Rightarrow o = +1$ ✓ (不更)
4. $(+1, +1, +1)$:
 $R = 0.4 + 0.3(+1) + 0.3(+1) = 1.0 \Rightarrow o = +1$ ✓ (不更)

第 2 轮零错误，算法收敛。

训练结果与检验

- 最终权重： $w_0 = 0.4, w_1 = 0.3, w_2 = 0.3$
- 决策边界： $0.4 + 0.3x_1 + 0.3x_2 = 0$

对四个点的最终判别：

- $(-1, -1) \Rightarrow R = -0.2 \Rightarrow o = -1$ (与 $d = -1$ 一致)
- $(-1, +1) \Rightarrow R = +0.4 \Rightarrow o = +1$ (一致)
- $(+1, -1) \Rightarrow R = +0.4 \Rightarrow o = +1$ (一致)
- $(+1, +1) \Rightarrow R = +1.0 \Rightarrow o = +1$ (一致)

结论：该数据线性可分，感知机在 2 个 epoch 内收敛到能正确分类所有样本的权重。



Perceptron – Training Algorithm

收敛定理

Convergence Theorem

The perceptron training rule will converge (finding a weight vector correctly classifies all training samples) within a finite number of iterations, **provided the training examples are linearly separable** and provided a sufficiently small η is used.

只要训练样本是线性可分的，并且使用了足够小的参数，那么感知机训练规则在有限的迭代次数内就会收敛（找到一个权重向量，能够正确地对所有训练样本进行分类）。

三、收敛成立的两个前提条件

1 数据线性可分 (linearly separable)

- 数据可以被一条直线（或超平面）完全分隔。

- 也就是说，存在一个 W^* 使得：

$$W^{*T}X > 0, \quad W^{*T}X < 0$$

- 如果数据本身不是线性可分的（例如 XOR 问题），则感知机永远不会收敛。

2 学习率足够小 (sufficiently small η)

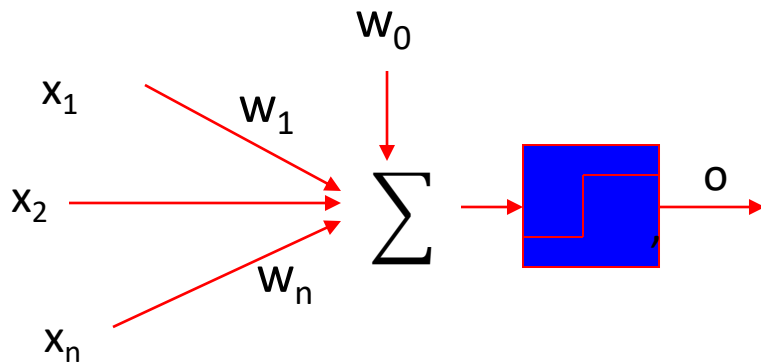
- 学习率 η 控制每次权重更新的步长；
- 若 η 太大，可能导致权重震荡或发散；
- 若 η 取较小值（如 0.01~0.1），算法可稳定收敛。

Adaptive Linear Element 自适应线性元件

(ADLINE)

是一种与感知机类似的神经元模型，但输出计算方式不同。

Perceptron

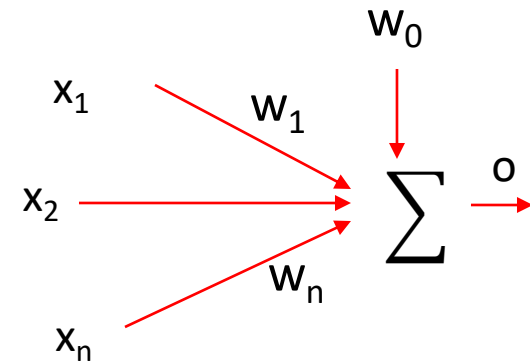


$$R = w_0 + \sum_{i=1}^n w_i x_i$$

$$o = \text{sign}(R) = \begin{cases} +1, & \text{if } R > 0 \\ -1, & \text{otherwise} \end{cases}$$

VS

ADLINE



$$o = w_0 + \sum_{i=1}^n w_i x_i$$

直接输出，不需要阈值函数

不经过符号函数，而是线性输出。输出是“连续值”，可以进一步用于误差分析和梯度下降优化。

感知机在非线性可分问题上会发散，
而 ADALINE 能通过最小化均方误差找到“最接近目标”的线性拟合解，因此在稳定性与实用性上优于感知机。

ADLINE VS Perceptron

Perceptron (感知机)

局限性：

当问题不是线性可分 (non-linearly separable) 时，感知机无法找到一条能完美分隔数据的直线（或超平面）。

后果：

算法会永远无法收敛 (fail to converge)，权重会在不同方向上不断更新、振荡，模型输出不稳定。

示例：

在 XOR 问题中，感知机无法找到任何能正确区分全部样本的线性边界。

- When the problem is not linearly separable, perceptron will fail to converge.

- ADLINE can overcome this difficulty by finding a best fit approximation to the target.

ADALINE (Adaptive Linear Neuron)

优势：

即使数据不是完全线性可分，ADALINE 仍能通过最小化误差 (error minimization) 找到最佳近似解 (best fit approximation)。

方法：

不强制要求所有样本完全正确分类；而是通过最小均方误差 (Mean Squared Error, MSE) 找到一条“误差最小”的分割线。

结果：

虽然不能完美分类，但输出总体更“平滑”、更接近目标；

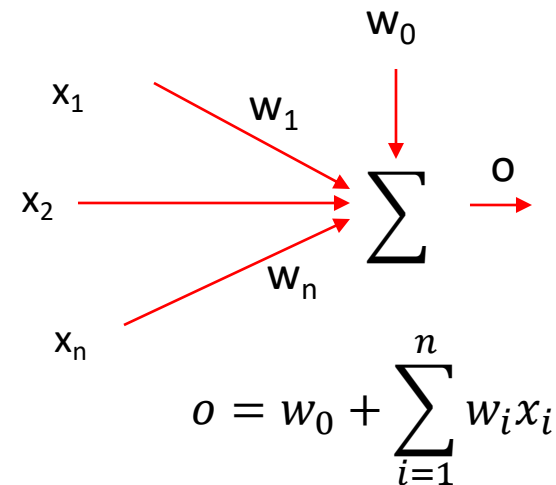
算法仍能稳定收敛（因为使用了连续可导的误差函数）。

The ADLINE Error Function

- We have training pairs $(X(k), d(k))$, $k = 1, 2, \dots, K$, where K is the number of training samples, the **training error** specifies the difference between the output of the ADLINE and the desired target.

- The error is defined as

$$E(W) \equiv \frac{1}{2} \sum_{k=1}^K (d(k) - o(k))^2$$



$o(k) = W^T X(k)$ is the output of presenting the training input $X(k)$

The ADLINE Error Function

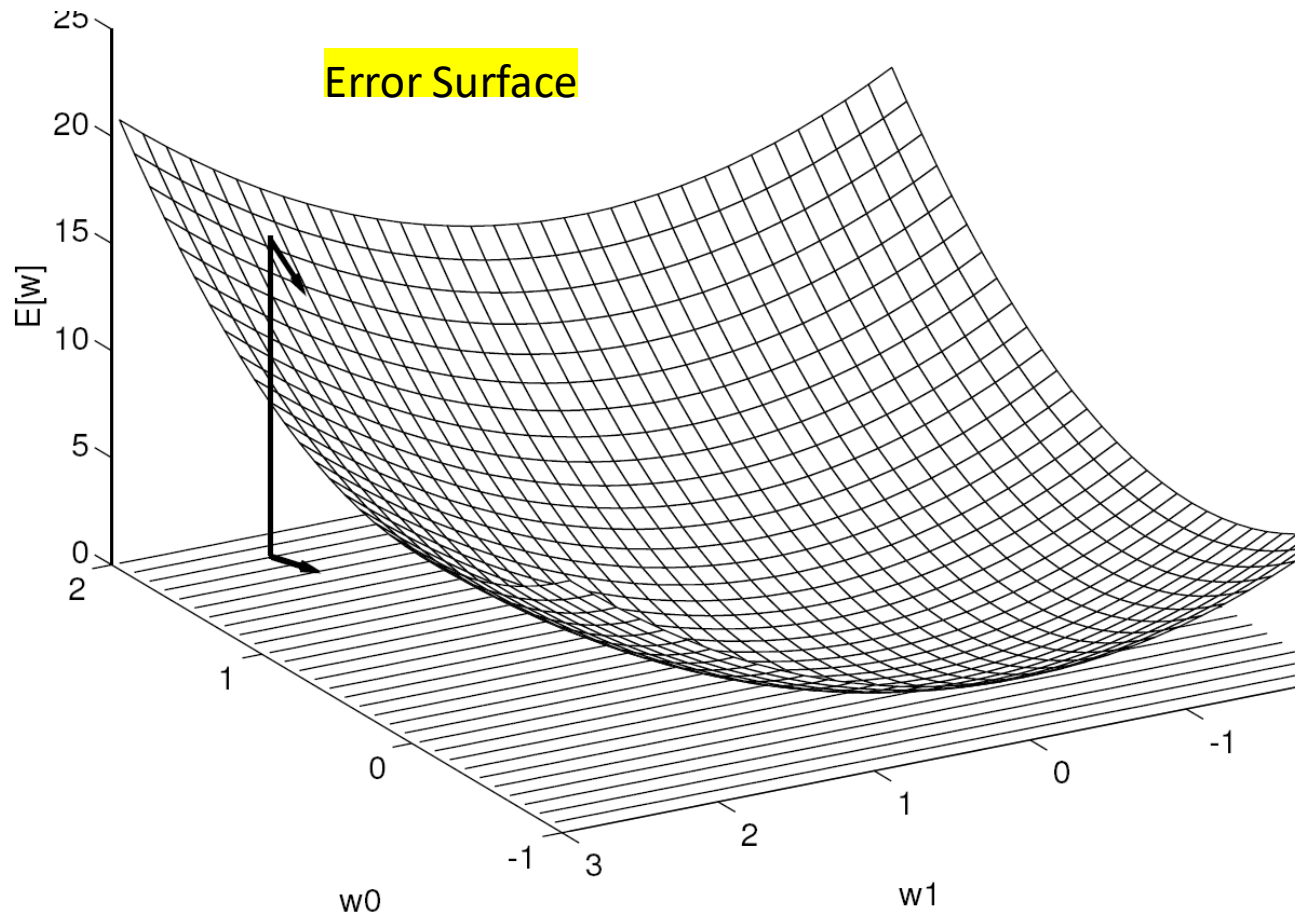
- The error is defined as

$$E(W) \equiv \frac{1}{2} \sum_{k=1}^K (d(k) - o(k))^2$$

误差越小，模型输出越接近目标

- The smaller $E(W)$ is, the closer is the approximation.
- We need to find W , based on the given training set, that minimizes the error $E(W)$. 也就是寻找一组最优权重 W ，使误差函数 $E(W)$ 达到最小值。

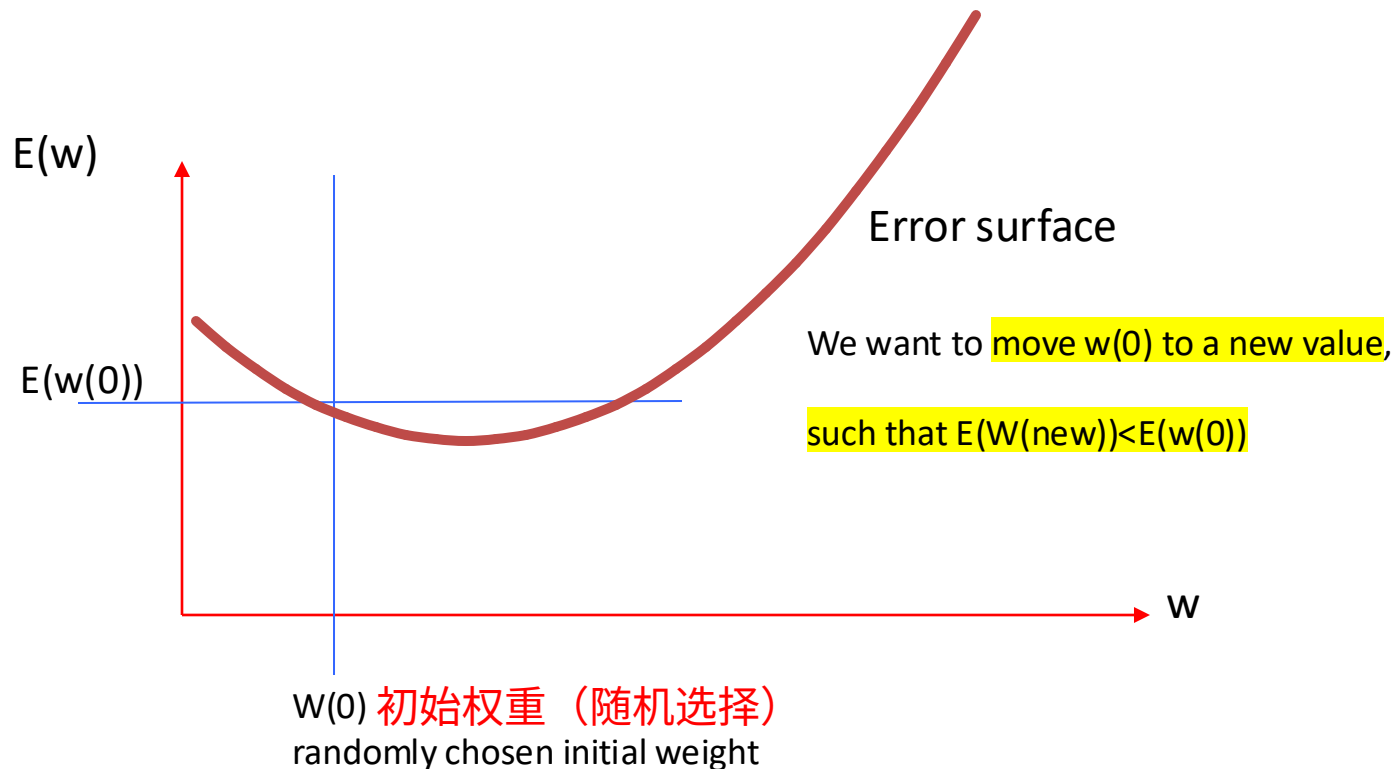
The ADLINE Error Function



The Gradient Descent Rule

An intuition 直观理解

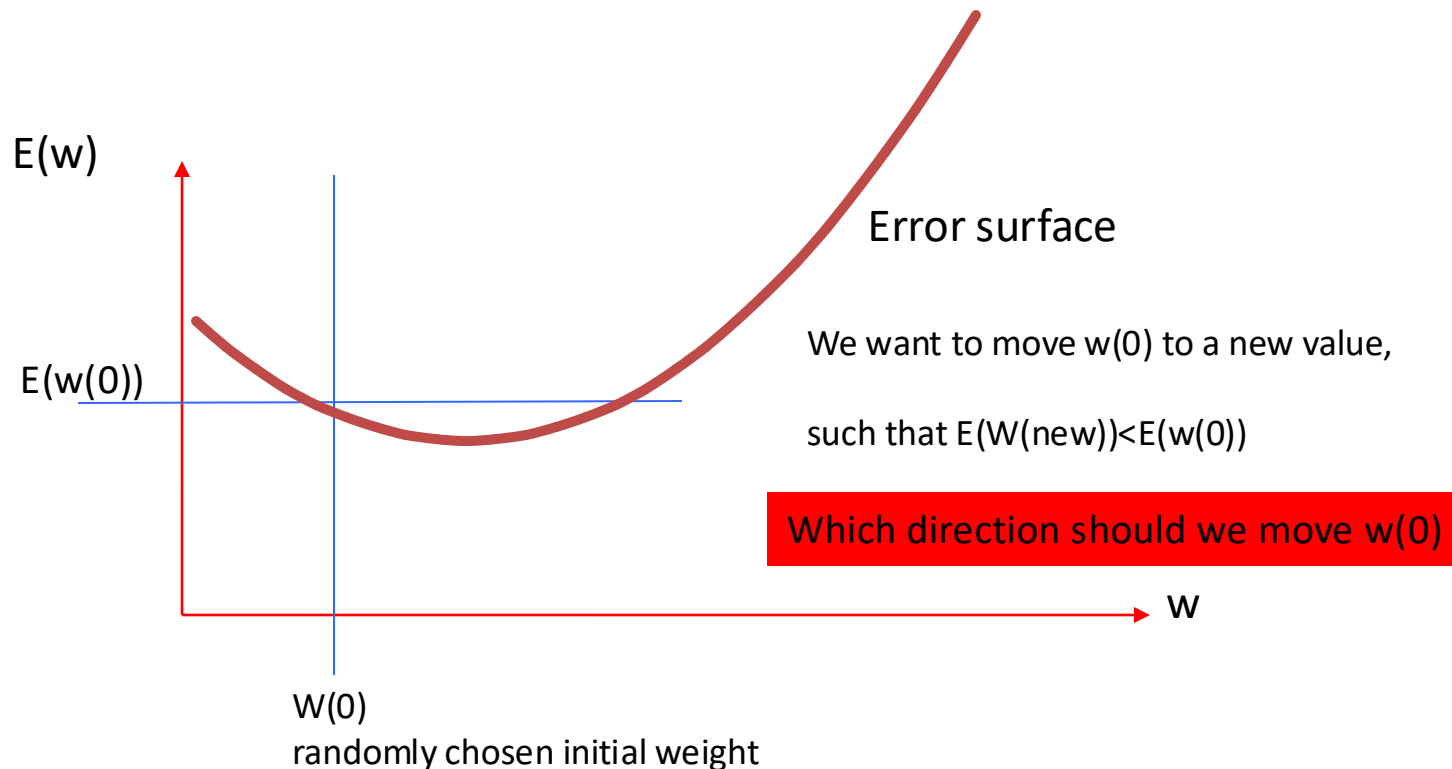
Before we formally derive the gradient decent rule, here is an intuition of what we should be doing.



The Gradient Descent Rule

An intuition

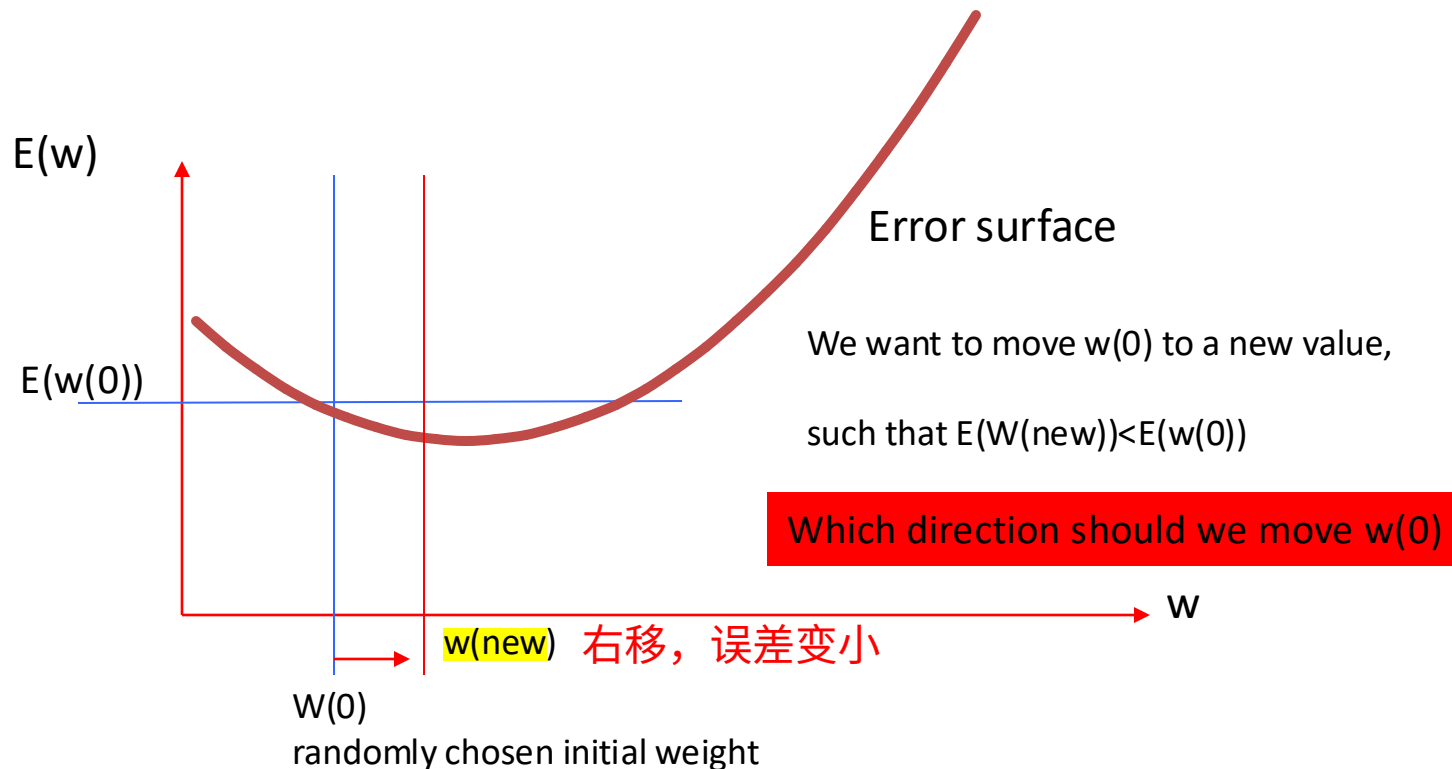
Before we formally derive the gradient decent rule, here is an intuition of what we should be doing.



The Gradient Descent Rule

An intuition

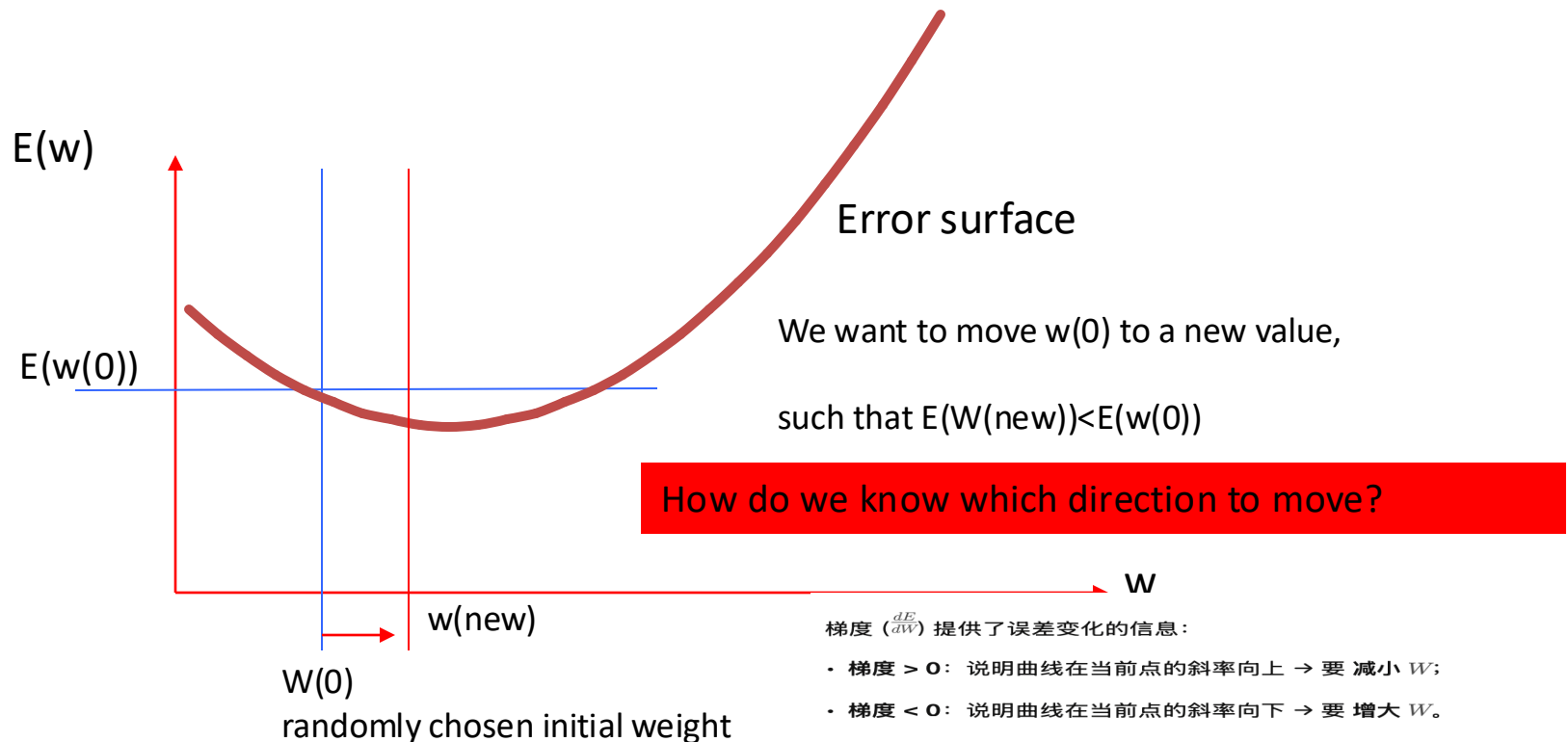
Before we formally derive the gradient decent rule, here is an intuition of what we should be doing.



The Gradient Descent Rule

An intuition

Before we formally derive the gradient decent rule, here is an intuition of what we should be doing.



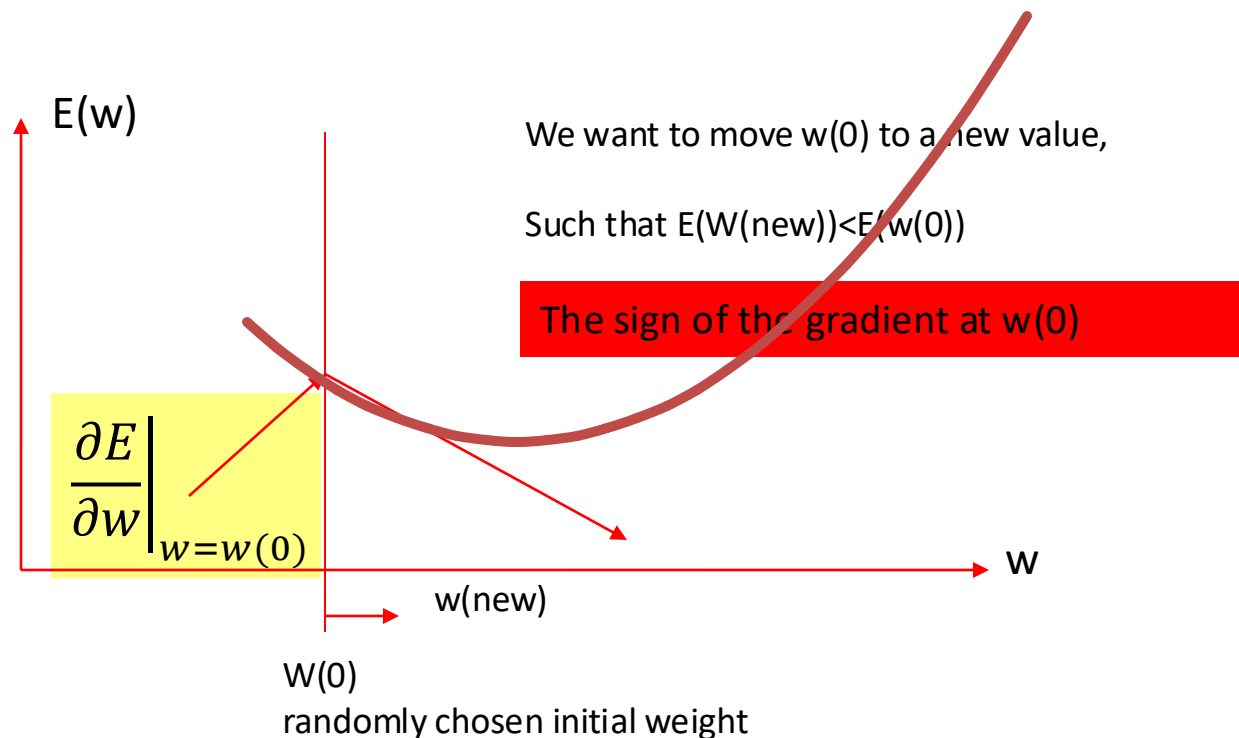
换句话说:

我们应该让权重沿着**梯度的反方向 ($-\nabla E(w)$)**移动, 因为那是让误差下降最快的方向。

The Gradient Descent Rule

An intuition

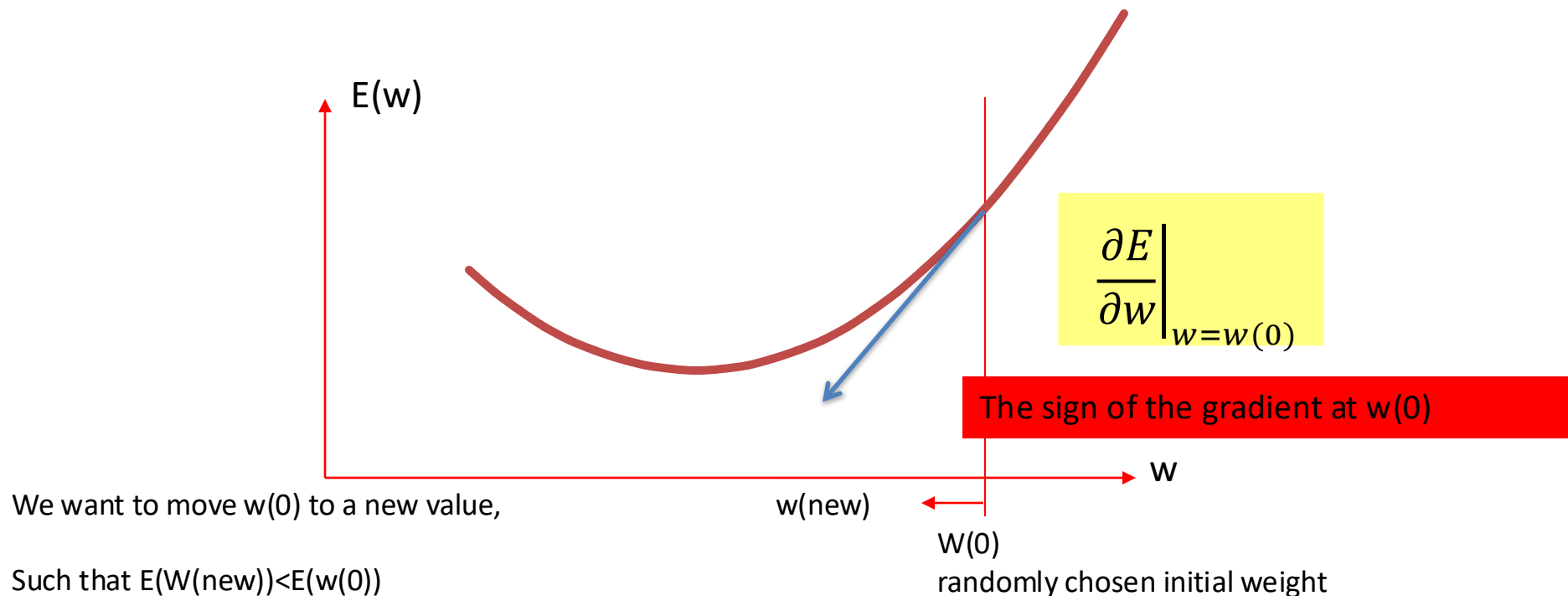
Before we formally derive the gradient decent rule, here is an intuition of what we should be doing.



The Gradient Descent Rule

An intuition

Before we formally derive the gradient decent rule, here is an intuition of what we should be doing.



The Gradient Descent Rule

An intuition

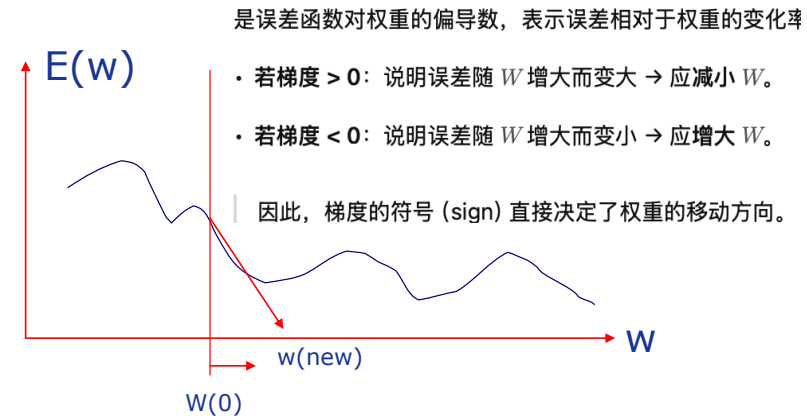
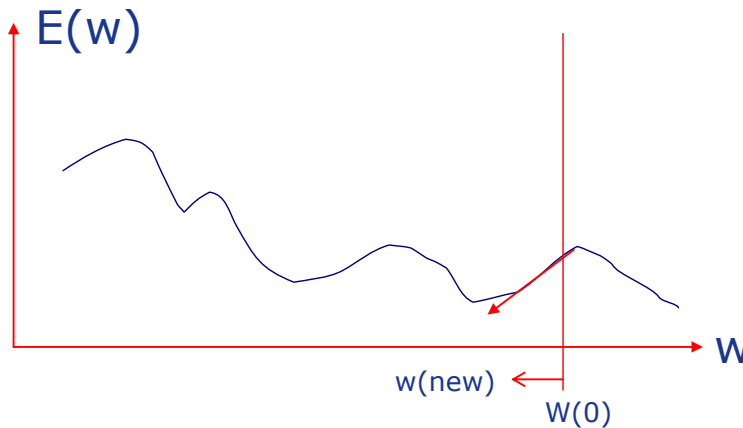
The intuition leads to the following rule:

更新权重

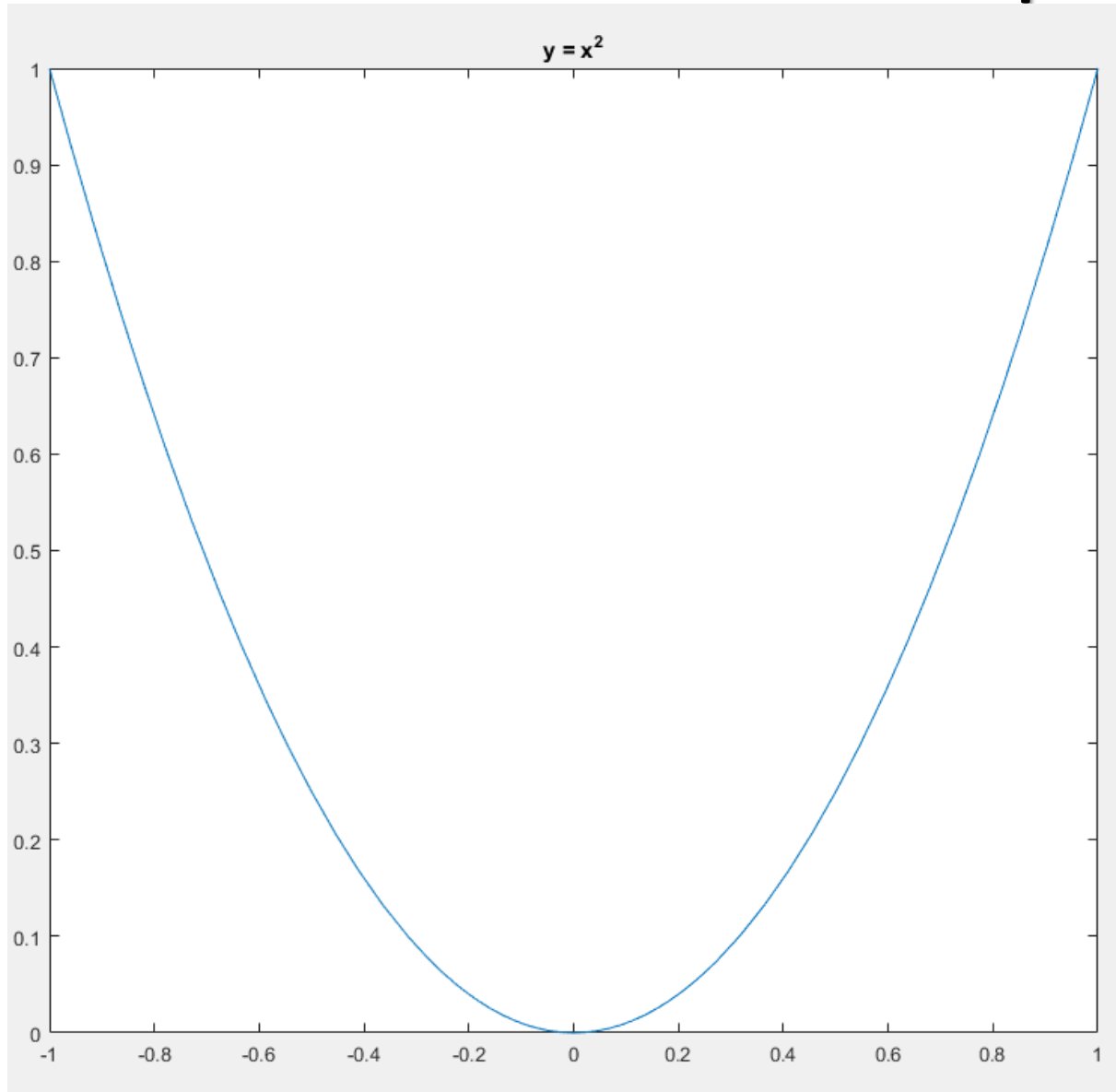
梯度

$$w(new) \leftarrow w(old) - \Delta \text{sign} \left(\left. \frac{\partial E}{\partial w} \right|_{w=w(old)} \right)$$

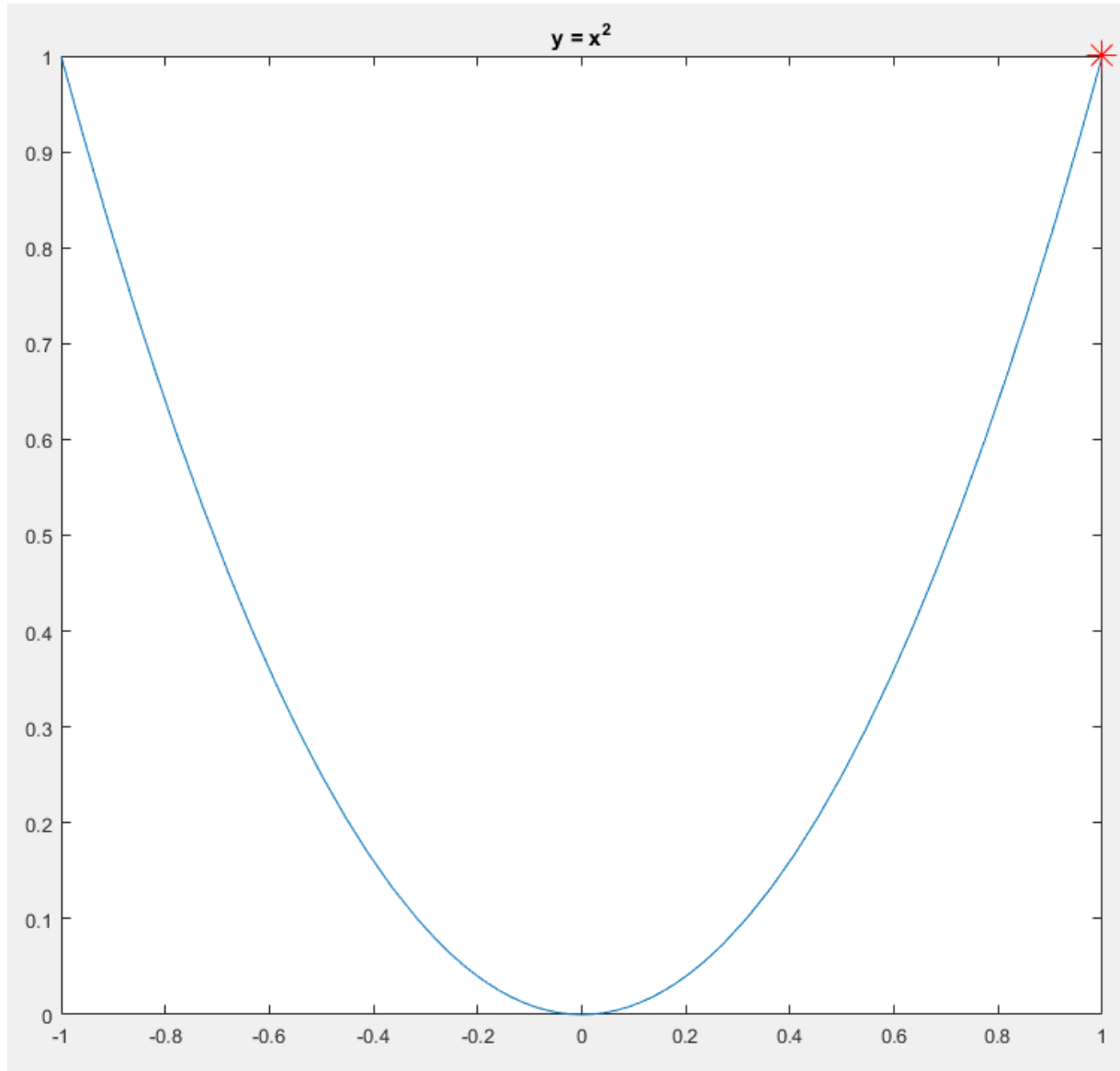
$\frac{\partial E}{\partial w}$



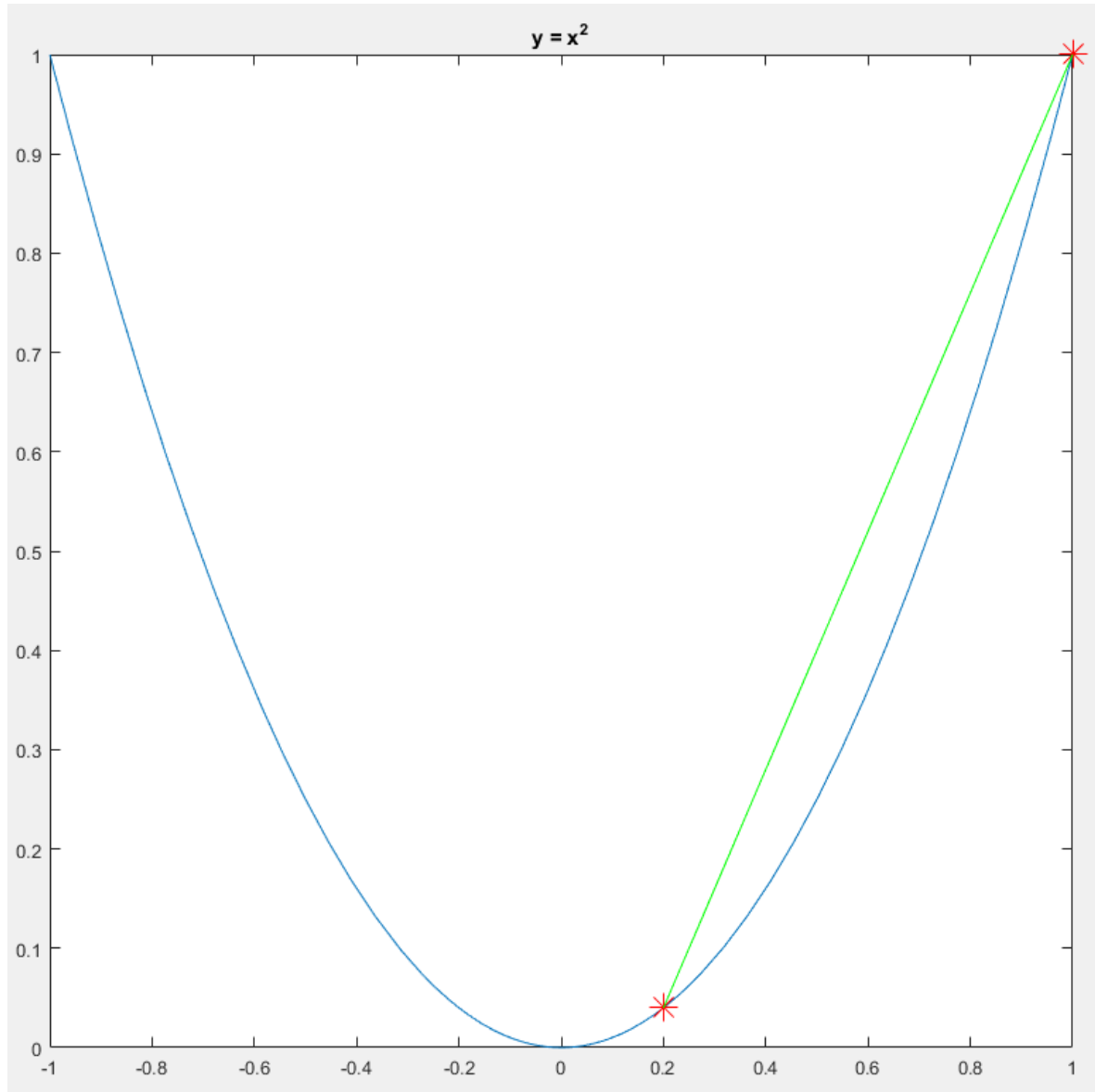
Gradient Descent Example



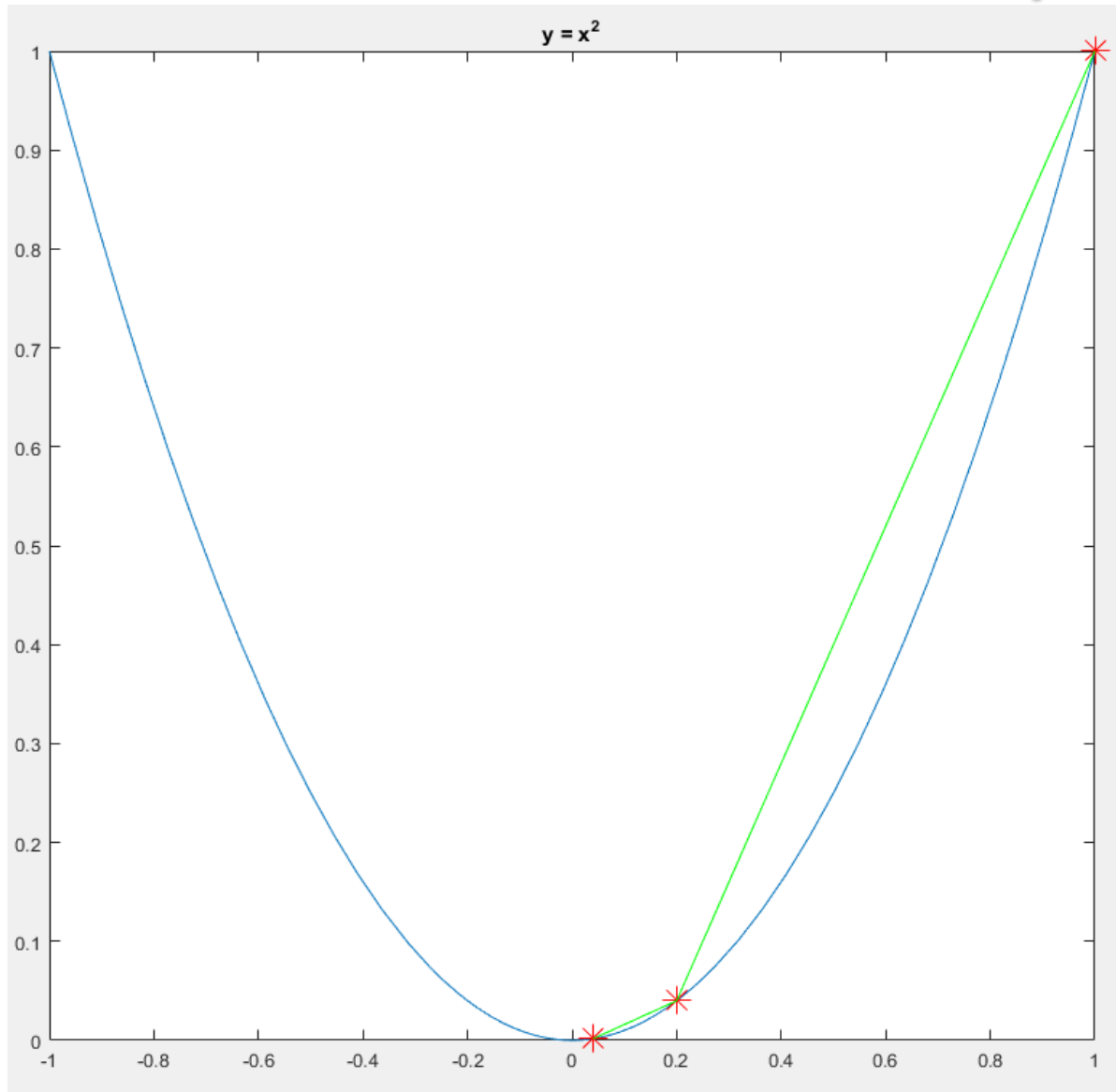
Gradient Descent Example



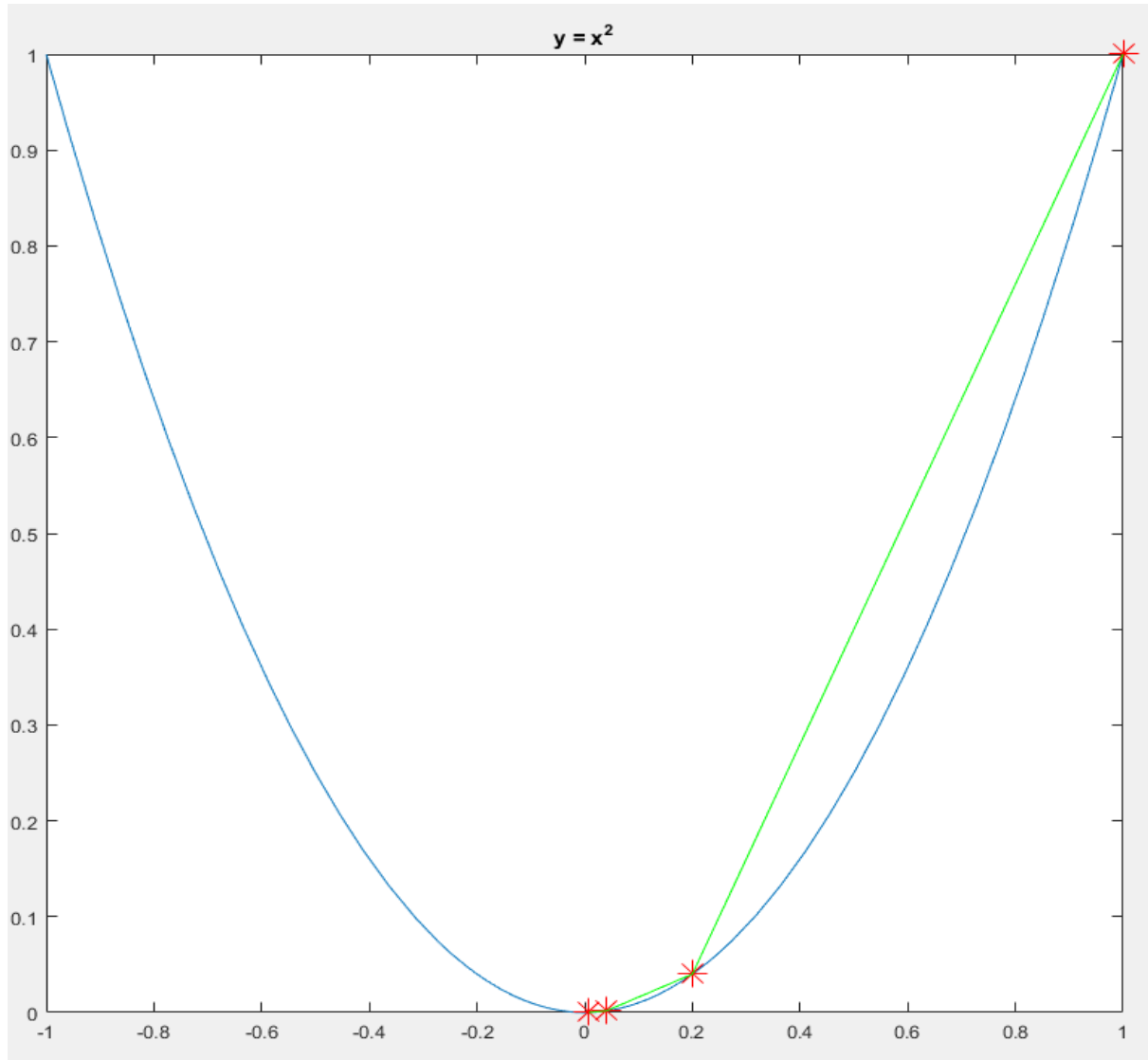
Gradient Descent Example



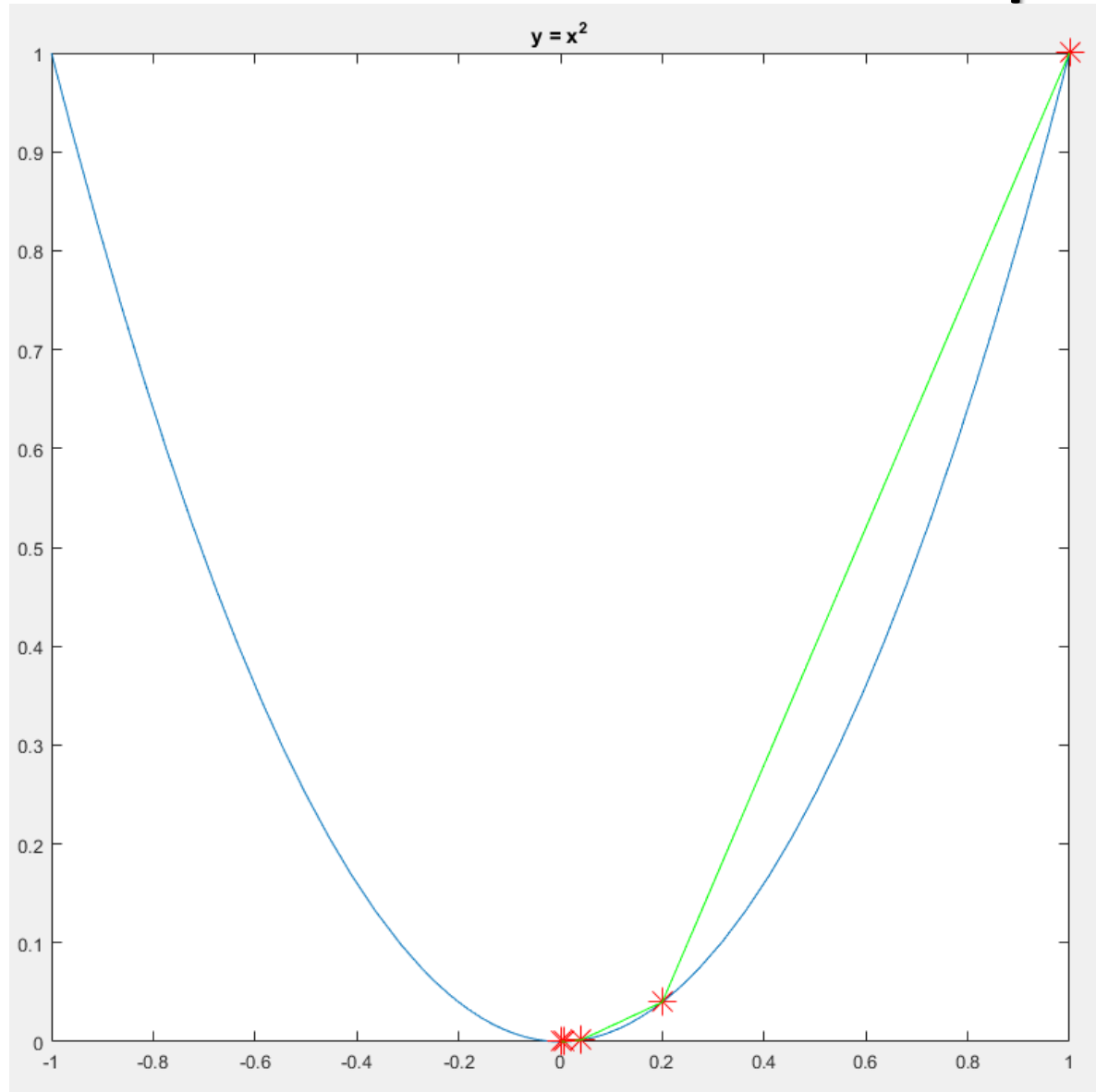
Gradient Descent Example



Gradient Descent Example



Gradient Descent Example



The Gradient Descent Rule

正梯度 上升最快方向；

负梯度 下降最快方向（即梯度下降法的方向）。

Formal Derivation of Gradient Descent

$$\nabla E(W) = \left[\frac{\partial E}{\partial w_1}, \frac{\partial E}{\partial w_2}, \dots, \frac{\partial E}{\partial w_n} \right]$$

梯度是一个向量 (vector)，其中每个分量表示误差函数对每个权重 w_i 的偏导数 (partial derivative)。

- The gradient of E is a vector, whose components are the partial derivatives of E with respect to each of the w_i 's

梯度指出了能使 E 值实现最大增幅的方向（即函数在某一方向上的变化速度（斜率））

- The gradient specifies the direction that produces the steepest increase in E . (i.e., speed of change (slope) of the function in one direction.)

该向量的负值则给出了 E 值实现最大降幅的方向。

- Negative of the vector gives the direction of steepest decrease in E .

页码	图示说明
第 32 页	展示初始误差曲线 $y = x^2$ ，此时尚未开始更新。最低点在 $x = 0$ 。
第 33 页	用红色星号标出初始点 $x_0 = 1$ (误差最大处)。
第 34 页	计算梯度 $\frac{dE}{dx} = 2x_0 = 2$ ，根据更新公式移动到新位置 $x_1 = x_0 - \eta \cdot 2x_0$ 。绿色线表示移动方向(向左)。
第 35 页	新点 x_1 已经靠近谷底，继续计算新的梯度(更小了)，再沿负梯度方向移动。
第 36 页	新点 x_2 更接近 0，误差显著下降。绿色线表示前后两步路径。
第 37 页	最终 x 接近 0 (最低点)，此时梯度 $\frac{dE}{dx} \approx 0$ ，算法收敛。

🧩 五、数值例子 (假设学习率 $\eta = 0.1$)

迭代步	x_{old}	梯度 $2x_{\text{old}}$	更新后 $x_{\text{new}} = x_{\text{old}} - 0.1(2x_{\text{old}})$	误差 $E(x) = x^2$
0	1.0	2.0	0.8	1.0
1	0.8	1.6	0.64	0.64
2	0.64	1.28	0.512	0.41
3	0.512	1.024	0.4096	0.26
4	0.4096	0.8192	0.3277	0.17
...	...	...	...	...
$\rightarrow \infty$	$\rightarrow 0$	$\rightarrow 0$	$\rightarrow 0$	$\rightarrow 0$

结果：经过多次更新后, x 趋近于 0，误差 $E(x)$ 趋近于最小值 0。

The Gradient Descent Rule

The gradient training rule is 梯度下降训练规则

$$W \leftarrow W - \eta \nabla E(W)$$

$$w_i \leftarrow w_i - \eta \frac{\partial E}{\partial w_i}$$

η is the training rate

学习率 η 控制更新步长:

学习率 η	结果
太小	收敛慢, 学习过程缓慢
太大	容易“跳过”最小点, 甚至震荡或发散
适中	稳定、平滑地收敛到误差最小点

- 梯度 $\nabla E(W)$: 表示误差上升最快的方向。
- 负梯度 $-\nabla E(W)$: 表示误差下降最快的方向。
- 因此每次更新的目标是让 $E(W)$ 逐步减小, 也就是让模型参数沿负梯度方向移动。

四、直观理解

- 如果当前梯度大 $\rightarrow$ 说明误差变化快 $\rightarrow$ 权重更新幅度较大;
- 如果当前梯度小 $\rightarrow$ 说明误差接近稳定 $\rightarrow$ 权重变化较小;
- 当梯度趋于 0 时 (即 $\frac{\partial E}{\partial w_i} \approx 0$), 模型已接近最优 (最小误差点)。

The Gradient Descent Rule

Gradient of ADLINE Error Functions

$$E(W) \equiv \frac{1}{2} \sum_{k=1}^K (d(k) - o(k))^2$$

$$\begin{aligned} \frac{\partial E}{\partial w_i} &= \frac{\partial E}{\partial w_i} \left(\frac{1}{2} \sum_{k=1}^K (d(k) - o(k))^2 \right) \\ &= \frac{1}{2} \sum_{k=1}^K \left(\frac{\partial E}{\partial w_i} (d(k) - o(k))^2 \right) \end{aligned}$$

$$= \frac{1}{2} \sum_{k=1}^K \left(2(d(k) - o(k)) \frac{\partial E}{\partial w_i} (d(k) - o(k)) \right)$$

这步在求导 $\frac{d}{dw_i}(a^2) = 2a \frac{da}{dw_i}$

$$= \sum_{k=1}^K \left((d(k) - o(k)) \frac{\partial E}{\partial w_i} \left(d(k) - w_0 - \sum_{i=1}^n w_i x_i(k) \right) \right)$$

在ADALINE中， $o = w_0 + \sum_{i=1}^n w_i x_i$
所以可以将 $o(k)$ 替换为上面的式子

$$= \sum_{k=1}^K ((d(k) - o(k))(-x_i(k)))$$

对上一括号内求导：

第1步：对括号求偏导（线性项对 w_i ）：

- $d(k)$ 与 w_i 无关，偏导为 0；
- w_0 与 w_i 无关（当 $i \geq 1$ ），偏导为 0；
- $\sum_{j=1}^n w_j x_j(k)$ 对 w_i 的偏导就是 $x_i(k)$ 。

$$= - \sum_{k=1}^K (d(k) - o(k)) x_i(k)$$

因此

$$\frac{\partial}{\partial w_i} \left[d(k) - w_0 - \sum_{j=1}^n w_j x_j(k) \right] = -x_i(k).$$

The Gradient Descent Rule

ADLINE weight updating using **gradient descent rule**

$$w_i \leftarrow w_i + \eta \sum_{k=1}^K (d(k) - o(k)) x_i(k)$$

上一页我们得到了误差函数对 w_i 的偏导：

$$\frac{\partial E}{\partial w_i} = - \sum_{k=1}^K (d(k) - o(k)) x_i(k)$$

代入梯度下降更新公式：

$$w_i \leftarrow w_i - \eta \frac{\partial E}{\partial w_i}$$

于是：

$$w_i \leftarrow w_i - \eta \left(- \sum_{k=1}^K (d(k) - o(k)) x_i(k) \right)$$

化简得到：

$$w_i \leftarrow w_i + \eta \sum_{k=1}^K (d(k) - o(k)) x_i(k)$$

The Gradient Descent Rule

Gradient descent training procedure

随机给每个权重一个较小的初始值，区间 $(-1, 1)$ ；选择一个合适的学习率，如0.2

- Initialise w_i to small values, e.g., in the range of $(-1, 1)$, choose a learning rate, e.g., $\eta = 0.2$

重复执行以下步骤，直到达到停止条件

- Until the termination condition is met, Do
 - For all training sample pair $(X(k), d(k))$, input the instance $X(k)$ and compute

$$\delta_i = - \sum_{k=1}^K (d(k) - o(k))x_i(k)$$

- For each weight w_i , Do

$$w_i \leftarrow w_i - \eta \delta_i$$

Batch Mode:

gradients accumulated over **ALL** samples first

Then update the weights

批处理模式：
首先将所有样本的梯度累加起来然后更新权重

Stochastic (Incremental) Gradient Descent

Also called online mode, Least Mean Square (LMS), Widrow-Hoff, and Delta Rule 也被称为在线模式、最小均方 (LMS)、维罗纳-霍夫以及德尔塔规则

- Initialise w_i to small vales, e.g., in the range of $(-1, 1)$, choose a learning rate, e.g., $\eta = 0.01$ (should be smaller than batch mode)
- Until the termination condition is met, Do
 - For EACH training sample pair $(X(k), d(k))$, compute

$$\delta_i = -(d(k) - o(k))x_i(k)$$

- For each weight w_i , Do

$$w_i \leftarrow w_i - \eta \delta_i$$

Online Mode:

Calculate gradient for
**EACH (or a SUBSET
of) samples**

在线模式：
为每个（或部分）样本计算梯度值然后更新权重

Then update the weights

特性	Batch Gradient Descent	Stochastic (Online) Gradient Descent
更新频率	每轮使用全部样本更新一次	每看一个样本就立即更新一次
学习率	可以较大 (如 $\eta=0.2$)	必须较小 (如 $\eta=0.01$)
收敛速度	慢但稳定	快但有波动
计算量	大 (一次需处理所有样本)	小 (逐步更新)
适用场景	数据量小、噪声低	数据量大、流式数据或在线学习

Training Iterations, Epochs

训练是一个反复迭代的过程；训练样本需要反复使用来进行训练。

- Training is an iterative process; training samples will have to be used repeatedly for training.
- Assuming we have K training samples $[(X(k), d(k)), k=1, 2, \dots, K]$; then an epoch is the presentation of all K sample for training once.
 - 一个 epoch (轮次) = 将这 K 个样本全部用于训练一次 (不论顺序)。
也就是说，模型完整地看过整个训练集一次，算作一个 epoch。
 - First epoch: Present training samples: $(X(1), d(1)), (X(2), d(2)), \dots (X(K), d(K))$
 - Second epoch: Present training samples: $(X(K), d(K)), (X(K-1), d(K-1)), \dots (X(1), d(1))$ 样本顺序可以 (也通常应该) 被随机打乱 (shuffled)，
这样可以避免模型对数据顺序产生依赖，从而提高泛化性能。
 - Note the order of the training sample presentation between epochs can (and should normally) be different. 请注意，在不同轮次中训练样本的呈现顺序是可以 (并且通常也应当) 有所不同的。
- Normally, training will take many epochs to complete.

Termination of Training

To terminate training, there are normally two ways

- 当训练达到预设的 epoch 数量时停止
When a pre-set number of training epochs is reached.
- 当平均误差 $E(W)$ 小于某个预设阈值时停
When the error is smaller than a pre-set value.

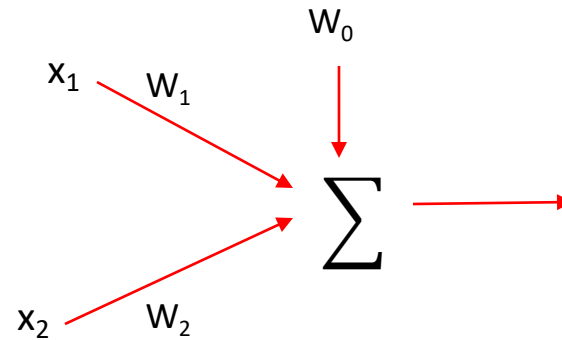
$$E(W) \equiv \frac{1}{2} \sum_{k=1}^K (d(k) - o(k))^2$$

梯度下降训练

Gradient Descent Training

Example

x1	x2	D
-1	-1	-1
-1	+1	+1
+1	-1	+1
+1	+1	+1



Initialization

$w_0(0)=0.1$; $w_1(0)=0.2$; $w_2(0)=0.3$;

$\eta=0.5$

样本 1:

$$x_1 = -1, \ x_2 = -1, \ d = -1$$

1 计算输出:

$$o(1) = w_0 + w_1x_1 + w_2x_2 = 0.1 + 0.2(-1) + 0.3(-1) = 0.1 - 0.2 - 0.3 = -0.4$$

2 误差:

$$e = d - o = (-1) - (-0.4) = -0.6 \quad \text{就是 (d-o) \times}$$

3 更新权重:

$$\begin{cases} w_0 \leftarrow 0.1 + 0.5(-0.6)(1) = 0.1 - 0.3 = -0.2 \\ w_1 \leftarrow 0.2 + 0.5(-0.6)(-1) = 0.2 + 0.3 = 0.5 \\ w_2 \leftarrow 0.3 + 0.5(-0.6)(-1) = 0.3 + 0.3 = 0.6 \end{cases}$$

◆ 更新后权重:

$$w_0 = -0.2, \quad w_1 = 0.5, \quad w_2 = 0.6$$

样本 2:

$$x_1 = -1, \ x_2 = +1, \ d = +1$$

1 输出:

$$o(2) = (-0.2) + 0.5(-1) + 0.6(+1) = -0.2 - 0.5 + 0.6 = -0.1$$

2 误差:

$$e = d - o = 1 - (-0.1) = 1.1$$

3 更新:

$$\begin{cases} w_0 \leftarrow -0.2 + 0.5(1.1)(1) = -0.2 + 0.55 = 0.35 \\ w_1 \leftarrow 0.5 + 0.5(1.1)(-1) = 0.5 - 0.55 = -0.05 \\ w_2 \leftarrow 0.6 + 0.5(1.1)(+1) = 0.6 + 0.55 = 1.15 \end{cases}$$

◆ 更新后:

$$w_0 = 0.35, \quad w_1 = -0.05, \quad w_2 = 1.15$$

样本 3:

$$x_1 = +1, \ x_2 = -1, \ d = +1$$

1 输出:

$$o(3) = 0.35 + (-0.05)(+1) + 1.15(-1) = 0.35 - 0.05 - 1.15 = -0.85$$

2 误差:

$$e = 1 - (-0.85) = 1.85$$

3 更新:

$$\begin{cases} w_0 \leftarrow 0.35 + 0.5(1.85)(1) = 0.35 + 0.925 = 1.275 \\ w_1 \leftarrow -0.05 + 0.5(1.85)(+1) = -0.05 + 0.925 = 0.875 \\ w_2 \leftarrow 1.15 + 0.5(1.85)(-1) = 1.15 - 0.925 = 0.225 \end{cases}$$

◆ 更新后:

$$w_0 = 1.275, \quad w_1 = 0.875, \quad w_2 = 0.225$$

样本 4:

$$x_1 = +1, \ x_2 = +1, \ d = +1$$

1 输出:

$$o(4) = 1.275 + 0.875(+1) + 0.225(+1) = 1.275 + 0.875 + 0.225 = 2.375$$

2 误差:

$$e = 1 - 2.375 = -1.375$$

3 更新:

$$\begin{cases} w_0 \leftarrow 1.275 + 0.5(-1.375)(1) = 1.275 - 0.6875 = 0.5875 \\ w_1 \leftarrow 0.875 + 0.5(-1.375)(+1) = 0.875 - 0.6875 = 0.1875 \\ w_2 \leftarrow 0.225 + 0.5(-1.375)(+1) = 0.225 - 0.6875 = -0.4625 \end{cases}$$

◆ 更新后:

$$w_0 = 0.5875, \quad w_1 = 0.1875, \quad w_2 = -0.4625$$

Further Reading

Chapter 4, T. M. Mitchell, Machine Learning,
McGraw-Hill International Edition, 1997